CE 356: Formal Spec and Verification
Term Project: Experiments and Extension
Dijkstra's Self-Stabilization Algorithm in TLA+

Joshua Yao | Spring 2026

1. Overview

This document records all TLC model-checking experiments on Dijkstra's self-stabilization algorithm, including a parameter sweep over N and K on the base algorithm, a series of extension experiments that modify process 0's increment step size, and formal invariant verification. The extension led to the discovery of a generalized correctness constraint that subsumes Dijkstra's original K >= N condition.

2. Base Algorithm Experiments

2.1 Design

Three families of experiments were run on the base algorithm (step size = 1):

- Vary N with K = N+1: observe how state space grows as the ring gets larger.
- Fix N=3, vary K below threshold (K < N): confirm self-stabilization breaks.
- Fix N=3, vary K above minimum: confirm extra headroom preserves correctness.

2.2 Results

N	K	K>=N?	Distinct States	Total States	Time (ms)	Result
2	3	YES	27 = 3^3	72	1550	**Pass**
3	2	NO	N/A	N/A	N/A	**FAIL**
3	3	YES (boundary)	81 = 3^4	270	1852	**Pass**
3	4	YES	256 = 4^4	896	2445	**Pass**
3	5	YES	625 = 5^4	2250	1807	**Pass**
4	5	YES	3125 = 5^5	13750	1957	**Pass**
5	6	YES	46656 = 6^6	248832	5271	**Pass**

Table 1: Base algorithm TLC results. Distinct states = K^(N+1) in all cases.

2.3 Analysis

Four key findings emerge from the base experiments:

State space is exactly K^(N+1). TLC exhaustively checks every possible starting configuration in all runs, confirming complete coverage of the state space.

The K < N failure is concrete. With N=3, K=2, TLC found a cycle showing the system loops indefinitely. Process 0 only generates values {0,1} before wrapping, which is not enough distinct values to fill a ring of 4 processes. The error trace was:

State 1: x = [0->0, 1->0, 2->1, 3->0]

State 2: x = [0->1, 1->0, 2->1, 3->0] process 0 fires, increments to 1

State 3: x = [0->1, 1->0, 2->1, 3->1] process 3 copies process 2

State 4: x = [0->1, 1->0, 2->0, 3->1] process 2 copies process 1

State 5: cycles back

The boundary case K = N works. N=3, K=3 passes because process 0 generates exactly 3 distinct values (0,1,2), just enough for a ring of 4 processes. Dijkstra's condition K >= N is tight.

Extra K headroom costs state space but preserves correctness. N=3 with K=5 checks 625 states vs 256 for K=4, with no benefit to correctness. The minimum K = N+1 is the most efficient choice.

3. Extension: Variable Step Size for Process 0

3.1 Motivation

The base algorithm has process 0 increment its counter by 1 modulo K. A natural question is whether self-stabilization is preserved if process 0 uses a different step size s. This tests the sensitivity of the coloring argument to process 0's behavior and potentially reveals a deeper generalization of Dijkstra's constraint.

3.2 TLA+ Modification

Only one line of the spec changes. The original Act0:

Act0 == x' = [x EXCEPT ![0] = (x[0] + **1)** % K]

The modified Act0 with step size s=2:

Act0 == x' = [x EXCEPT ![0] = (x[0] + **2)** % K]

All other definitions remain identical, making this a clean controlled experiment.

3.3 Generalized Constraint

The initial prediction was that self-stabilization requires gcd(s, K) = 1. However, experiments revealed the correct constraint is:

K / gcd(s, K) >= N

The quantity K / gcd(s, K) is the number of distinct values that process 0 visits before its counter repeats. This number just needs to be >= N, not exactly equal to K. This generalizes Dijkstra's original constraint: when s=1, gcd(1,K)=1 always, so K/gcd = K >= N, which is exactly Dijkstra's condition.

3.4 Extension Results

N	K	step	K/gcd(s,K)	K/gcd>=N ?	Distinct States	Result	Predicted
3	4	2	4/2 = 2	2 >= 3? NO	N/A	**FAIL**	FAIL
3	5	2	5/1 = 5	5 >= 3? YES	625 = 5^4	**Pass**	PASS
3	6	2	6/2 = 3	3 >= 3? YES	1296 = 6^4	**Pass**	FAIL*
4	5	2	5/1 = 5	5 >= 4? YES	3125 = 5^5	**Pass**	FAIL*
4	7	2	7/1 = 7	7 >= 4? YES	16807 = 7^5	**Pass**	PASS

*Table 2: Extension results with step size s=2. * = initial prediction was wrong, corrected by refined constraint.*

3.5 Analysis

The refined constraint K / gcd(s, K) >= N correctly predicts all five experimental outcomes. The two cases marked * are where the initial gcd(s,K)=1 prediction was wrong:

- N=3, K=6: predicted **FAIL** (gcd=2, thought only 3 values), actually **PASS** because 3 values is enough for N=3.
- N=4, K=5: predicted **FAIL** (gcd=1 but thought 5 values insufficient), actually **PASS** because 5 >= 4.

The generalized constraint is strictly weaker than gcd(s,K)=1 AND K>=N, meaning the modified algorithm admits more valid (N, K, s) combinations than initially predicted, making it more flexible in practice.

4. Formal Invariant Verification

4.1 The Invariant

Beyond the liveness property **SelfStabilization**, I identified and verified a safety invariant:

AtLeastOnePrivileged == NumPrivileged >= 1

This states that in every reachable state, at least one process is privileged. Unlike SelfStabilization, which is a liveness property (something eventually happens), this is a safety property (something bad never happens). Specifically, it guarantees the system can never deadlock.

4.2 Theoretical Justification

The invariant holds by a simple case analysis on any state:

- Case 1: x[0] = x[N]. Then process 0 is privileged by definition.

- Case 2: x[0] not equal to x[N]. Then process 0 is not privileged. But the chain x[0], x[1], ..., x[N] has x[0] not equal to x[N], so somewhere adjacent values must differ. That process is privileged.

This argument holds for any values of x, regardless of K, step size, or execution history. It is a structural property of the privilege rules themselves.

4.3 TLC Verification Results

N	K	Step	Distinct States	SelfStabilization	AtLeastOnePrivileged
3	4	1	256 = 4^4	**Pass**	**Pass**
3	2	1	16 = 2^4	**FAIL**	**Pass**
3	4	2	256 = 4^4	**FAIL**	**Pass**

Table 3: Invariant verification results. Yellow rows are broken configurations where SelfStabilization fails but AtLeastOnePrivileged still holds.

4.4 Significance

The most important finding is in the yellow rows of Table 3. Even when the system fails to self-stabilize (K too small, or step size incompatible with K), the invariant AtLeastOnePrivileged still holds. This means:

- The system never gets permanently stuck regardless of the parameters.
- Self-stabilization failure is not due to deadlock but due to the system cycling forever without converging.
- AtLeastOnePrivileged is a deeper, unconditional guarantee baked into the privilege rule's design.

This distinction between liveness (self-stabilization) and safety (no deadlock) is a key insight of this project. The algorithm's design cleanly separates these concerns: the privilege rules guarantee safety unconditionally, while the parameter constraints (K >= N, or, more generally, K/gcd(s,K) >= N) guarantee liveness.

5. Summary of All Findings

- The base algorithm self-stabilizes for all N and K >= N, as verified exhaustively by TLC up to N = 5.
- Self-stabilization fails for K < N: TLC finds concrete cycles proving non-convergence.
- Modifying process 0's step size from 1 to s preserves self-stabilization if and only if K / gcd(s, K) >= N.
- Dijkstra's original K >= N constraint is a special case of the generalized constraint with s = 1.
- AtLeastOnePrivileged is an unconditional structural invariant that holds for all parameter choices and is verified by TLC.

- Self-stabilization failure is always due to cycling, never deadlock. Safety and liveness are cleanly separated.

These findings will be incorporated into the final report along with formal theorem statements and a discussion of implications for the design of self-stabilizing distributed systems.